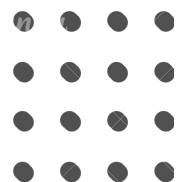




VMES Grinds

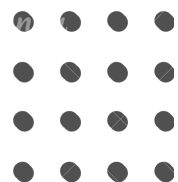




VMES Grinds

Pilot Program - Week 2 - Day 2

Hash Maps • Strings • Bucketing





Leetcode: 49 Group Anagrams



Moe Myint Mo San



49. Group Anagrams

Solved

Medium



Topics



Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.



Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- `1 <= strs.length <= 104`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.



Problem Explanation Thought Process





Initial Thought Process

Linear iteration through the array of words.

We need a way to detect anagrams efficiently.

Anagrams share the same:

- sorted characters, or
- character frequencies

We need a signature → map all words with the same signature into one group.



Thought Process

Anagram definition:

Words are anagrams if they contain the same frequency of each character.

Goal →

Create a unique signature for each word using:

- Sorted characters (simple)
- Character frequency (optimized)

Use a Hash Map:

signature → list of anagrams



Algorithm Evolution Overview



Approach	Time	Space	Core Idea
1▯ Sorted String Key	$O(n \cdot k \log k)$	$O(n \cdot k)$	Sort each string as the group key
2▯ Character Count Key	$O(n \cdot k)$	$O(n \cdot k)$	26-length frequency signature



Sorted String Key





Idea:

Sort each string.

All anagrams have identical sorted strings.

Example:

```
eat → aet  
tea → aet  
ate → aet
```

Use the sorted string as the hash map key:

"aet" → ["eat", "tea", "ate"]

This builds the foundation → "Group by identical signature".



```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        res = defaultdict(list)
        for s in strs:
            sortedS = ''.join(sorted(s))
            res[sortedS].append(s)
        return list(res.values())
```

Analysis:

Time: $O(n \cdot k \log k)$ Space: $O(n \cdot k)$

Good intuition builder.

Becomes slow when strings are long.

- Where n is the number of strings and k is the length of the longest string -



Sorted Keys → Hash Map Buckets



```
strs =  
["eat", "tea", "tan", "ate", "nat", "bat"]
```

Sorted keys:

```
"eat" → "aet"  
"tea" → "aet"  
"tan" → "ant"  
"ate" → "aet"  
"nat" → "ant"  
"bat" → "abt"
```

Hash Map Buckets:

```
"aet" → ["eat", "tea", "ate"]  
"ant" → ["tan", "nat"]  
"abt" → ["bat"]
```



Character Frequency Signature (Optimized Approach)





Idea:

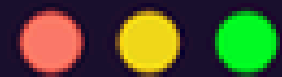
Words are anagrams if each letter appears the same number of times.

Build a frequency list of 26 integers:

Example for "eat":

a:1, e:1, t:1 → (1,0,0,0,1,0,,,1)

This tuple becomes the group key.



```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        res = defaultdict(list)
        for s in strs:
            count = [0] * 26
            for c in s:
                count[ord(c) - ord('a')] += 1
            res[tuple(count)].append(s)
        return list(res.values())
```



"tea"

t:1, e:1, a:1 → (1,1,0,0,...0,1)

"eat"

e:1, a:1, t:1 → (1,1,0,0,...0,1)

→ SAME KEY → Same bucket

Index:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
"eat":	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
"tea":	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0



Why This Works

- Faster because counting is $O(k)$
- Signature size always 26
- No sorting
- Great for long strings or large datasets

Analysis:

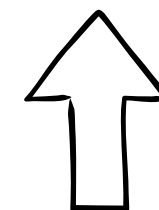
Time: $O(n \cdot k)$ Space: $O(n \cdot k)$

- Optimal solution for interviews.

– Where n is the number of strings and k is the length of the longest string –



Extra Study Material –Visualizations–





Complexity Comparison



Method	Time	Space	Notes
Sorted String	$O(n \cdot k \log k)$	$O(n \cdot k)$	Easiest, clean signature
Count Signature	$O(n \cdot k)$	$O(n \cdot k)$	Optimal, fixed 26-size key



Key Takeaway





Core Concepts

- Anagrams share a unique signature
- Hash Maps group identical signatures
- Two signatures:
 - Sorted string → simple & intuitive
 - Character frequency → optimal
- Pattern generalizes to:
 - String hashing
 - Grouping by features
 - Classification problems



Thank You !

